

Bug Report & Action Plan

SubTeach Mobile (React Native Expo)

Prepared by: QA team

Date: 28 June 2026

Version: 2.0 (post-pull audit)

Based on commit: 622729f (branch `main`)

Delivered to: Development team

About this document

This document contains 83 bugs confirmed through five rounds of independent source-code analysis. All findings come from reading and statically analysing the code — **no automated test suite has been run** because none yet exists (see `TEST_PLAN.md` for the plan to build one).

The QA team has **no privilege to modify source code**. All fixes are the dev team's decision and execution. This document documents findings, recommends fixes, and proposes a sprint priority.

Two critical regressions to fix today, before anything else:

- **Bug #38** — Credit card payment is completely broken (crashes on every submit) due to `insertInvoice` signature mismatch introduced by the recent rewrite.
- **Bug #70** — Teacher's "Confirm Attendance" and "Unable to Attend" buttons crash the JS thread because they import functions that don't exist (`unableAttendance` , `confirmAttendance`).

Both are reproducible on first user tap with no special setup. Both block central workflows. Both should be P0 hotfixes.

Executive Summary

Analysis result: **83 confirmed bugs** across the codebase. Distribution:

- CRITICAL** 11 bugs — security, data leakage, or guaranteed runtime crashes
- HIGH** 27 bugs — user-visible breakage or workflow blocker
- MEDIUM** 27+ bugs — correctness / testability
- LOW** 17+ bugs — code quality / cleanup

Two of the CRITICAL bugs are **regressions introduced by the recent rewrite** (Bugs #38 and #70). They block the credit-card payment flow and the teacher attendance-confirmation flow respectively. Either is reproducible by a single user tap. These should be hotfixed before any other work.

One bug (Bug #6 — broken status badge mapping) was **fixed** by the recent pull and is documented here for completeness; the fix introduced Bug #12, a smaller variant of the same type-drift class.

CRITICAL bug summary

#	Title	Location
1	User.role string/numeric mismatch → cross-school data leakage	contexts/SessionContext.tsx:326
2	Password persisted plaintext to AsyncStorage	types/index.ts:6 , lib/storage.ts:14
3	Bearer tokens logged to OS log on every request	lib/api.ts:33-34
10	Split-brain backend URL (vizzion vs kreatifa)	lib/api.ts:4 vs AuthContext.tsx:58/226/280
15	Teacher document edits hit school endpoint (cross-role)	app/edit-document.tsx:192
16	Console logs leak upload responses, suburb data, full booking objects	multiple sites
17	Suburb selection broken in teacher profile (value/label mismatch)	app/(teacher-tabs)/profile.tsx:228-250
38	REGRESSION: insertInvoice signature mismatch — pay flow broken	app/(school-tabs)/invoices.tsx:220
39	Stripe paymentMethod.id and full error bodies logged	app/(school-tabs)/invoices.tsx:216-230
40	Full user object logged on every route change	app/_layout.tsx:25
70	REGRESSION: unableAttendance , confirmAttendance imported but don't exist — buttons crash	app/session-detail.tsx:23-30

Recommendation: Start with Sprint 1 (the 11 CRITICAL bugs, total estimated effort ~5 hours). These are the security and crash-on-tap issues that should be fixed before the next release regardless of feature priorities.

CRITICAL Bugs — Full Detail

Bug #1 — Cross-school data leakage (role type inconsistency)

Severity: **CRITICAL** Effort: ~30 min

Location: `contexts/SessionContext.tsx:326` , `contexts/AuthContext.tsx:83-114`

Cause

The type `UserRole` is declared as numeric `9 | 10`. The auth layout file compares numerically (correctly). However, `SessionContext.tsx:326` compares to a string:

```
if (user.role === 'school') {  
  return mapped.filter((b) => b.schoolId === user.id);  
}  
return mapped;
```

Since `user.role` is always a number (9 or 10), `=== 'school'` is **always false**. The filter never runs.

Business impact

Every user with the school role sees confirmation bookings belonging to **every school in the system**, not just their own. This is a cross-tenant data leak — not a cosmetic issue. School A sees booking details from school B, C, D, etc.

Fix: `user.role` must have one shape, decided in `AuthContext` at login time. Choose numeric (`9 | 10`) since that's what `UserRole` declares. Normalise every API response into that shape. Then change `SessionContext.tsx:326` to `if (user.role === 10)`, or better, add a helper `isSchool(user)` in one place (e.g. `lib/roles.ts`) and use it everywhere.

Bug #2 — Password persisted to AsyncStorage in plaintext

Severity: **CRITICAL** Effort: ~30 min

Location: `types/index.ts:6` , `lib/storage.ts:14-16` , `contexts/AuthContext.tsx:160-161`

Cause

The `User` interface declares `password` as a required field. `storage.saveUser` serialises the whole object. `register()` explicitly writes `password: userData.password`.

Business impact

AsyncStorage on iOS lives in unencrypted SQLite; on Android in a plaintext file. Readable by: another process on a rooted/jailbroken device, anyone with an unencrypted device backup (iTunes, `adb backup` , MDM export), or any forensic tool with physical access. Compliance exposure for an education-sector app under Australian Privacy Principles (`.com.au` backend).

Fix: Remove `password` from the `User` interface, delete the `password: userData.password` line in `register()` , and only persist `{ id, email, name, role, role_id, token, termsAccepted, ... }` . The token is the only secret the client needs. Ideally move even the token to `expo-secure-store` .

Bug #3 — Bearer tokens logged to OS log on every request

Severity: **CRITICAL** Effort: ~5 min

Location: `lib/api.ts:33-34` , `contexts/AuthContext.tsx:126`

Cause

```
// lib/api.ts:33-34
console.log('AUTH TOKEN:', authToken);
console.log('STORAGE TOKEN:', await getStorageToken());

// contexts/AuthContext.tsx:126
console.log('LOGIN TOKEN:', data.token, 'normalizedRole=', ...);
```

Business impact

`console.log` in React Native is **not a no-op in production**. Hermes still executes the call; the string is emitted to:

- Android `logcat` — readable by other apps with `READ_LOGS` , MDM agents, OEM crash collectors
- iOS unified log (`Console.app` / `idevicesyslog`) — captured in sysdiagnose archives, exported by MDMs
- Crash reporter breadcrumbs (Sentry/Bugsnag/Crashlytics) — shipped with every crash report to third-party SaaS

A bearer token in the log is session-takeover material. Combined with Bug #2, no password is needed to impersonate the user.

Fix: Delete the three logging lines. For dev ergonomics, wrap any future debug logs in `if (__DEV__)` , but never log the token even in dev — dev logs end up in screenshares, Slack threads, and Loom recordings.

Bug #10 — Split-brain backend URL (`vizzeon.com` VS `kreatifa.com`)

Severity: **CRITICAL** Effort: ~1 hour

Location: `lib/api.ts:4` vs `contexts/AuthContext.tsx:58, 226, 280`

Cause

The recent pull migrated the auth domain from `vizzeon.com` to `kreatifa.com`, but only inside `AuthContext`. The api wrapper still points at the old host:

```
// lib/api.ts:4 (UNCHANGED – old domain)
const BASE_URL = 'https://teacher-relief.vizzeon.com/api';

// contexts/AuthContext.tsx:58, 226, 280 (CHANGED – new domain)
await fetch('https://teacher-relief.kreatifa.com/api/auth/login', ...)
await fetch('https://teacher-relief.kreatifa.com/api/auth/logout', ...)
await fetch('https://teacher-relief.kreatifa.com/api/auth/reset_password', ...)
```

Business impact

The app talks to **two different backends in the same session**. Login / logout / reset go to `kreatifa`; every other API call (profile, sessions, notifications, documents, invoices) goes to `vizzeon`. If `vizzeon` is decommissioned, the app breaks after login. If it still exists but is stale, users see inconsistent state. This is the direct consequence of the existing Bug #7 (hardcoded URLs); the duplicated pattern hid the incomplete migration.

Fix: Unify under `expo-constants` (move the URL to `app.json` `extra.apiBaseUrl` or to an `EXPO_PUBLIC_API_BASE_URL` env var), resolve once at module load, and make sure all four sites point at the same host.

Bug #15 — Teacher document edits hit the SCHOOL endpoint

Severity: **CRITICAL** Effort: ~30 min

Location: `app/edit-document.tsx:192`

Cause

```
await editSchoolDocument(documentId, formData); // ← unconditional, regardless of role
```

The screen loads either teacher or school documents based on `user?.role`, but `handleSubmit` always calls `editSchoolDocument`. The role check at line 195 only affects the *redirect destination*, not the API call.

Business impact

A teacher (role 9) editing their own document hits a school endpoint. Backend either rejects (best case), edits a school record by ID collision, or applies school-scoped permissions to teacher data. Cross-role data corruption risk.

Fix: Branch on `user?.role` and call `editTeacherDocument` for teachers (or pass role to a generic service).

Bug #16 — Console logs leak full API responses, upload metadata, and booking objects

Severity: **CRITICAL** Extends: Bug #3

Location: multiple sites (see below)

Sites

- `app/(school-tabs)/profile.tsx:187, 350, 354, 377` — `console.log('UPLOAD RES:', res)`, `console.log('SUBURB RES:', res)`
- `app/(teacher-tabs)/profile.tsx:203, 326, 374, 378` — same pattern
- `app/session-confirmation-detail.tsx:125` — `console.log(booking)` on every render
- `app/create-session.tsx:207, 221, 242, 287, 442, 494` — six `console.error` with API response objects (may contain teacher PII like email/phone)

Business impact

Beyond bearer tokens (Bug #3), the device log now contains signed photo URLs, full upload responses, booking PII (teacher names, schedules, IDs), and suburb-API responses. Same capture surfaces as Bug #3 (logcat, iOS unified log, crash-reporter breadcrumbs).

Fix: Strip all `console.log` / `console.error` in production, or wrap in `if (__DEV__)`.

Bug #17 — Suburb selection broken in teacher profile (value/label type mismatch)

Severity: **CRITICAL** Effort: ~1 hour

Location: `app/(teacher-tabs)/profile.tsx:228-230, 246-250, 557`

Cause

```
const selectedSuburb = useMemo(() => {
  return suburbsForState.find((item) => item.value === location_suburb_id) || null;
}, ...);
```

`location_suburb_id` from backend is a string, but `item.value` is the raw unstringified `item.id`. `===` fails when types differ → `selectedSuburb` always `null` on first load → postcode never auto-fills, suburb appears blank. The school version at line 197 does `String(item.id)` correctly — this is an inconsistency between the two screens.

Business impact

Teachers cannot see or change their suburb. Postcode field stays empty. Save submits with the wrong values.

Fix: Mirror the school screen's pattern — `String(item.id)` when building `suburbsForState`, compare with stringified IDs everywhere.

Bug #38 — **REGRESSION** `insertInvoice` signature mismatch — pay flow completely broken

Severity: **CRITICAL** Effort: ~30 min

Location: `app/(school-tabs)/invoices.tsx:220` vs `lib/services/school.ts:210`

Cause

```
// invoices.tsx – caller (only 2 args)
await insertInvoice(paymentTarget.id, paymentMethod.id);

// school.ts – declared signature (requires 3 args)
export async function insertInvoice(
  invoiceId: string,
  paymentMethod: string,
  payload: { fileUri: string; mimeType: string; fileName: string }
)
```

The service immediately accesses `payload.fileUri` to build FormData → `TypeError: Cannot read properties of undefined (reading 'fileUri')` on every submit.

Business impact

Every credit-card payment attempt fails. User sees the generic "Failed to submit payment" toast. The recent rewrite changed the service from "upload proof of payment" to "Stripe paymentMethod flow" but the caller never updated.

Fix: Either make `payload` optional in the service signature (if the Stripe flow doesn't upload proof), or add a file picker for proof of payment in the modal so the third argument can be supplied.

Bug #39 — Stripe `paymentMethod.id` and full error bodies logged to console

Severity: **CRITICAL** Extends: Bugs #3, #16

Location: `app/(school-tabs)/invoices.tsx:216-218, 227-230`

```
console.log('PAYMENT TARGET:', paymentTarget.id);
console.log('PAYMENT METHOD:', paymentMethod.id); // pm_xxx – payment instrument ID
console.log('PAYMENT ERROR:', JSON.stringify(err?.response?.data, null, 2));
```

Business impact

Stripe `pm_...` IDs appear in `logcat`, iOS unified log, crash-reporter breadcrumbs. The error JSON dump can contain backend internals.

Fix: Strip these logs or gate them behind `if (__DEV__)`. Never include `paymentMethod.id` even in dev.

Bug #40 — Full `user` object logged on every route change

Severity: **CRITICAL** Extends: Bug #3

Location: `app/_layout.tsx:25`

```
console.log('[RootNavigation] run route check', { user, isLoading, segments });
```

Business impact

Every navigation logs the entire user record — email, role, token-adjacent fields, verification status. Runs on every route change, including hot deep-links.

Fix: Remove `user` from the payload, or wrap the entire log in `__DEV__`.

Bug #70 — **REGRESSION** Imports two functions that don't exist — attendance buttons crash

Severity: **CRITICAL** Effort: ~30 min

Location: `app/session-detail.tsx:23-30`, used at `:206`, `:217`

Cause

```
import {
  acceptSession, declineSession, checkInSlot, checkOutSlot,
  unableAttendance, confirmAttendance, // ← neither exported from teacher.ts
} from '@lib/services/teacher';
```

Neither symbol exists in `lib/services/teacher.ts` (or anywhere else in the repo). At runtime both resolve to `undefined`. The first tap on either button throws `TypeError: undefined is not a function`.

Business impact

The new attendance-confirmation UX (introduced in the +76-line change to this file) is **completely non-functional**. Teachers cannot confirm or decline attendance, which gates the rest of the session lifecycle (check-in cannot proceed without confirmation in some paths).

Fix: Add the two service functions (pointing at `/teacher/session/confirm_attendance` and `/teacher/session/unable_attendance` or whatever the real endpoints are), or remove the imports and the two buttons until the backend endpoints exist.

HIGH Bugs (27)

Each entry summarises the bug, location, and recommended fix. Full reproduction steps and code excerpts are available in QA's working memory if needed.

#	Title	Location	Fix summary
4	AppNotification field drift (<code>is_read</code> vs <code>read</code>); unread badge never decreases for API notifications	<code>types/index.ts:107-108</code> , <code>contexts/SessionContext.tsx:270-316</code>	One source of truth — pick API shape or add adapter at repository boundary
5	TeachingSession <code>snake_case</code> type vs <code>camelCase</code> runtime; locally-created sessions render blank	<code>types/index.ts:65-86</code> , <code>contexts/SessionContext.tsx</code> (multiple sites)	Pick <code>snake_case</code> as canonical, adapter at form-input boundary, enable <code>noUncheckedIndexedAccess</code>
7	<code>BASE_URL</code> hardcoded, no dev/staging/prod separation	<code>lib/api.ts:4</code> , <code>contexts/AuthContext.tsx:58/223/277</code>	Move to <code>expo.extra.apiUrl</code> via <code>expo-constants</code> , resolve once
8	"Sarah Johnson" test helper runs in production data loader	<code>contexts/SessionContext.tsx:115-129</code>	Delete the block; move seed data to <code>lib/mockData.ts</code> mock mode only
14	<code>ApiInvoicesRepository</code> uses relative URLs that fail in RN (if mode flipped to <code>api</code>)	<code>lib/repositories/invoicesRepository.ts:53,63,87</code>	Route through the <code>api</code> wrapper; never use relative URLs in RN
18	<code>verification_*</code> / <code>active</code> read from <code>User</code> but absent from the type	both profile files (<code>:122-123</code> , <code>:140-141</code>)	Add fields to <code>User</code> type or read from API response object
19	<code>verifStatus == 2</code> blocks logout — user trapped in app	both profile files (<code>:307-310</code> , <code>:329-333</code>)	Remove the logout block; magic <code>2</code> needs a named constant
20	<code>updateUser</code> imported but never called after profile save — context stays stale	both profile files	Call <code>updateUser(...)</code> after successful save, or refetch profile
21	<code>request_ids: [...active, ...deleted]</code> concatenated; backend can't distinguish	<code>app/create-session.tsx:381</code>	Send <code>deleted_request_ids</code> as a separate field; keep <code>request_ids</code> aligned with <code>schedules</code>
22	<code>booking.booking_status == '1'</code> magic comparison; Leave Review CTA likely never appears	<code>app/session-confirmation-detail.tsx:214</code>	Compare to the correct <code>SessionStatus</code> value (e.g. <code>'completed'</code>); use a named constant
23	<code>setProfileImage</code> typed <code>string</code> can become	both profile files	<code>setProfileImage(res.photo ?? school?.profileImage ??</code>

#	Title	Location	Fix summary
	<code>undefined</code> ; clears photo on race		<code>''</code>)
24	<code>/uploads</code> substring detection of "existing server file" — false positives drop new uploads	<code>app/edit-document.tsx:175</code>	Track an explicit <code>isNewFile</code> flag set when <code>pickFile</code> runs
41	Reset-password guard duplicated AND bypassable via substring match	<code>app/_layout.tsx:21-24, 36-38</code>	Strict path match (<code>pathname === '/reset-password'</code>); add to <code>publicRoutes</code>
42	Unrecognised role leaves user stuck on <code>/login</code> with only a console message	<code>app/_layout.tsx:48-105</code>	Normalise role to numeric once; show fallback UI or force-logout for unknown roles
43	<code>verification_status</code> defaults to <code>1</code> (verified) when missing — FAIL OPEN security gate	<code>app/_layout.tsx:52</code>	Default to <code>0</code> (unverified) — fail closed
44	<code>formatCurrency</code> is just <code>`\${amount}`</code> — no decimals, no thousands, no null guard, unit ambiguity	<code>app/(school-tabs)/invoices.tsx:39-41</code>	Use <code>Intl.NumberFormat('en-AU', {style:'currency', currency:'AUD'})</code> with <code>Number(amount) 0</code> ; clarify unit (dollars vs cents)
45	Invoice filter UI missing <code>waiting</code> and <code>rejected</code> options even though those statuses exist	<code>app/(school-tabs)/invoices.tsx:35-37</code>	Add <code>'waiting'</code> and <code>'rejected'</code> to <code>FilterStatus</code> and <code>STATUS_FILTERS</code>
46	<code>keyExtractor={item => item.id}</code> but <code>id</code> is numeric — React warning + potential key collision	<code>app/(school-tabs)/invoices.tsx:325</code>	<code>keyExtractor={({item}) => String(item.id)}</code>
47	<code>getInvoiceData()</code> no client-side school filter (same pattern as Bug #1)	<code>app/(school-tabs)/invoices.tsx:115</code>	Add defensive filter <code>item.invoice.filter(inv => inv.school_id === user?.id)</code> ; clear state on user change
60	School registration missing Step 2 entirely (no document upload, terms acceptance, integrity checkbox)	<code>app/register-school.tsx</code> (only 254 lines vs teacher's 659)	Add Step 2 mirroring <code>register-teacher.tsx:331-420</code> ; confirm with backend whether schools are intentionally exempt
71	Teacher screen calls school-only endpoint <code>getDashboardData()</code> regardless of role	<code>app/session-detail.tsx:21, 44-50</code>	Branch on <code>user.role</code> ; use a teacher endpoint for role 9
72	<code>fetchData</code> has no <code>try/catch</code> ; failure leaves	<code>app/session-detail.tsx:44-54</code>	Wrap in <code>try { ... } catch { notify(...) } finally {</code>

#	Title	Location	Fix summary
	"Loading..." spinner forever		<code>setLoading(false) }</code>
74	No double-tap guard on Accept / Decline / Check In / Check Out / Confirm Attendance / Unable buttons	<code>app/session-detail.tsx:201-266, 303-316</code>	Add a <code>busy</code> state guard for all six handlers
75	No authorisation check on Accept / Decline (only Check-In/Out gate by <code>teacher_user_id</code>)	<code>app/session-detail.tsx:301</code> vs <code>:196-197</code>	Apply the same teacher-id guard to Accept/Decline visibility
78	"Check Out" button shown without verifying user has checked in (status-machine hole)	<code>app/session-detail.tsx:249-266</code>	Gate on <code>slot.check_in_time != null && slot.check_out_time == null</code>

MEDIUM Bugs (27+)

#	Title	Location
9	<code>AuthContext</code> bypasses the <code>api</code> wrapper at three sites (<code>fetch</code> directly)	<code>contexts/AuthContext.tsx:58, 223, 277</code>
12	<code>resolveSessionStatus</code> returns <code>'attended'</code> / <code>'unattended'</code> not in <code>SessionStatus</code> type	<code>components/ui/AppPrimitives.tsx:113-114</code>
13	<code>active</code> , <code>verification_status</code> , <code>verification_logs</code> written to <code>User</code> but missing from interface	<code>contexts/AuthContext.tsx:119-121</code>
25	Web Blob upload missing MIME → backend rejects valid PDFs/images on web	<code>app/edit-document.tsx:177-179</code>
26	Silent <code>catch {}</code> blocks in <code>edit-document</code> — root cause unreachable for QA	<code>app/edit-document.tsx:108, 153, 207</code>
27	Schedule slots never validated for <code>endTime > startTime</code> or overlap	<code>app/create-session.tsx:337-339</code>
28	<code>getTeachers</code> race — <code>useEffect</code> + inline call fire concurrently	<code>app/create-session.tsx:254, 431</code>
29	<code>SchoolProfile.schoolName</code> vs API <code>school_name</code> — auto-fill silently fails	<code>app/create-session.tsx:294</code>
30	Private-request <code>teacher_id</code> accepts raw user-typed string when <code>teacherInfo.id</code> null	<code>app/create-session.tsx:472-474</code>
31	State-reset placed outside <code>finally{} → failure leaves button stuck loading forever</code>	<code>app/session-confirmation-detail.tsx:74, 104</code>
32	Profile fetch <code>useEffect</code> has no <code>AbortController</code> — race between focus events	both profile files
33	Email validation is just <code>.includes('@')</code> — accepts <code>@</code> , <code>a@</code> , <code>@b</code>	both profile files
48	Missing <code>return</code> after <code>router.replace</code> chain in <code>_layout.tsx</code> — redirect flicker / race	<code>app/_layout.tsx</code>
49	<code><Stack key={user?.role}></code> remounts navigator on every login/logout — blows away all screen state	<code>app/_layout.tsx:118</code>
50	<code>refreshData()</code> sets <code>loading=true</code> on pull-to-refresh — wrong indicator	<code>app/(school-tabs)/invoices.tsx:112-133</code>
51	<code>openLogs(id)</code> has no try/catch and no loading state	<code>app/(school-tabs)/invoices.tsx:175-181</code>
52	Logs-modal <code>RefreshControl</code> refetches invoice list (wrong handler) instead of logs	<code>app/(school-tabs)/invoices.tsx:424-432</code>
53	<code>useEffect</code> deps include <code>user</code> but body doesn't read it; state not cleared on user change	<code>app/(school-tabs)/invoices.tsx:135-139</code>
54	Invoice status compared as magic strings <code>'1'/'2'/'3'</code> in 4 sites → Pay button on already-paid invoice → double payment	<code>app/(school-tabs)/invoices.tsx</code> (multiple)
55	<code>verification_status</code> cast via <code>as any</code> , not on <code>User</code> type	<code>app/_layout.tsx:52</code>

#	Title	Location
61	<code>console.log('ERROR SUBURB:', err)</code> leaks raw axios error in registration	<code>app/register-school.tsx:86-88</code>
62	Suburb options keyed on label instead of id; name+postcode collision sends wrong <code>location_id</code>	<code>app/register-school.tsx:100-115</code>
63	Address not validated; email is <code>.includes('@')</code> ; phone has no validation	<code>app/register-school.tsx:117-131</code>
65	<code>useElapsedTime</code> runs <code>setInterval</code> every 1s on every SessionCard regardless of status	<code>components/SessionCard.tsx:49-61</code>
66	SessionCard lacks defensive reads; renders "undefined undefined" for missing teacher names	<code>components/SessionCard.tsx:106-126</code>
69	SessionCard reads <code>session.is_confirm</code> not declared on <code>TeachingSession</code>	<code>components/SessionCard.tsx:75</code>
73	session-detail <code>useEffect</code> no cleanup / <code>AbortController</code>	<code>app/session-detail.tsx:52-54</code>
76	<code>resolveSessionStatus(session.status, ...)</code> but the field is <code>request_status</code>	<code>app/session-detail.tsx:74</code>
77	<code>sessions.find((s) => s.request_id == id)</code> mixed string/number; no missing-id validation	<code>app/session-detail.tsx:57</code>
79	No check-in time-window enforcement (teacher can check in days early or late)	<code>app/session-detail.tsx:236-246</code>
80	<code>console.log('decline res:', res)</code> leaks server response	<code>app/session-detail.tsx:131</code>

LOW Bugs (17+)

#	Title	Location
11	Duplicate file <code>app/(school-tabs)/documents.txt</code> byte-for-byte copy of <code>documents.tsx</code>	delete the <code>.txt</code> file
34	<code>alert('...')</code> instead of <code>notify(...)</code> in both profile files (alert IS polyfilled — not a crash, just inconsistent UX)	both profile files
35	<code>loadingRating</code> misnamed; stores rating value, not a loading flag	<code>app/(teacher-tabs)/profile.tsx:84</code>
36	WWCC constants imported but picker commented out — incomplete feature in main	<code>app/create-session.tsx:93-128</code>
37	Loose <code>==</code> comparisons throughout — hide string/number type drift	<code>app/session-confirmation-detail.tsx</code> , both profile files
56	<code> </code> precedence makes name-fallback unreachable — "null null" shown	<code>app/(school-tabs)/invoices.tsx:362</code>
57	Hardcoded "Demo Card (Test Mode) — 4242 4242 4242 4242" UI text visible in production	<code>app/(school-tabs)/invoices.tsx:493-497</code>
58	Stripe <code>publishableKey="pk_test_51Te5Yg..."</code> hardcoded	<code>app/_layout.tsx:138</code>
59	Dead comments and informal log labels (<code>'ROLE GA JELAS'</code> , etc.) in <code>_layout.tsx</code>	<code>app/_layout.tsx</code>
64	Form state not persisted; backgrounding the app wipes everything	<code>app/register-school.tsx:40-49</code>
67	Hardcoded <code>en-AU</code> locale and no timezone in date formatter	<code>components/SessionCard.tsx:19-24</code>
68	Dead <code>amountRow</code> / <code>amountLabel</code> / <code>amount</code> styles never used	<code>components/SessionCard.tsx:201-218</code>
81	Loose <code>==</code> on <code>user?.role == 9 / 10</code> masks Bug #1 pattern	<code>app/session-detail.tsx:92, 93</code>
82	Unused <code>isSchool</code> symbol; dead comments; unused <code>errorText</code> style (no error UI at all)	<code>app/session-detail.tsx</code>
83	<code>await fetchData()</code> before <code>router.replace</code> on decline → UX hang on slow networks	<code>app/session-detail.tsx:130-135</code>

Action Plan

Recommended split into sprints based on severity and technical dependencies. Total estimated effort: **~16-20 hours of developer time** for all CRITICAL and HIGH bugs combined.

Sprint 0 — P0 Hotfix (this week, ~1 hour)

The two regressions introduced by the recent rewrite. Both crash the app on first user tap. No backend changes needed to verify the fix.

- **Bug #38** (~30 min) — fix `insertInvoice` signature; payment flow restored
- **Bug #70** (~30 min) — add or remove `unableAttendance` / `confirmAttendance` ; attendance buttons restored

Sprint 1 — Security & Data Leak (this week, ~4 hours)

The remaining 9 CRITICAL bugs. Must be fixed before the next release.

- **Bug #3, #16, #39, #40** (~30 min total) — strip all token / PII `console.log` calls; wrap any kept dev logs in `__DEV__`
- **Bug #1** (~30 min) — normalise `user.role` to numeric in `AuthContext` ; replace string comparison with helper
- **Bug #2** (~30 min) — remove `password` from `User` type and from persistence
- **Bug #10** (~1 hour) — unify backend URL via `expo-constants` ; resolve once
- **Bug #15** (~30 min) — branch on role in `edit-document` ; teachers call teacher endpoint
- **Bug #17** (~1 hour) — fix suburb selection in teacher profile (mirror school pattern)

QA verification: after these deploy to staging, QA will write 11 regression tests (one per CRITICAL bug) to lock the fix.

Sprint 2 — User-Facing Quality (next sprint, ~10-12 hours)

The 27 HIGH bugs grouped by surface for efficiency:

- **Type drift cluster** (~4 hours) — Bugs #4, #5, #18, #29 — fix the `snake_case` vs `camelCase` contracts on `TeachingSession`, `AppNotification`, `SchoolProfile`, and `User`. Enable `noUncheckedIndexedAccess`.
- **Status machine cluster** (~2 hours) — Bugs #22, #45, #46, #54, #78 — replace magic-string comparisons with typed constants; cover all status values in the UI.
- **Profile / save flow** (~2 hours) — Bugs #19, #20, #23, #43 — fix logout-block, call `updateUser` after save, default `verification_status` to fail-closed, defensive photo state.
- **Session lifecycle holes** (~2 hours) — Bugs #21, #24, #41, #42, #71, #72, #74, #75 — request-id integrity, file-reuse detection, auth-routing edges, error states, idempotency, authorisation gates.
- **Currency & environment** (~1 hour) — Bugs #7, #44, #47 — proper currency formatting, environment-aware URL, defensive school filter.
- **Registration gap** (~2 hours) — Bug #60 — add Step 2 to `register-school.tsx` (after confirming with backend that schools should have it).
- **Dead code / fixes** (~30 min) — Bugs #8, #14 — remove Sarah Johnson helper, route invoices repository through api wrapper.

Sprint 3 — Cleanup & Refactor (scheduled, ~3-4 hours)

- **Bug #9** (~1 hour) — route `AuthContext` auth calls through the `api` wrapper
- **27 MEDIUM bugs** — error handling, `AbortControllers`, validation, dead state — addressable as time allows
- **17 LOW bugs** — code quality, dead code, magic strings — pick up when the surface is touched anyway

QA prerequisites from the dev team

After Sprint 1 is complete, QA will begin building the automated test suite (full plan in `TEST_PLAN.md`). To do that, QA needs from the dev team:

1. OpenAPI / Postman spec for endpoints `/auth/*`, `/teacher/*`, `/school/*`
2. Decision on canonical `User.role` shape (recommendation: numeric `9 | 10`) — same fix as Bug #1
3. Decision on canonical `TeachingSession` shape (recommendation: `snake_case` matching the backend) — same fix as Bug #5
4. Staging accounts: 1 teacher + 1 school, with sessions seeded in each lifecycle state
5. Staging backend URL separate from production (achieved automatically by fixing Bug #7 / #10)
6. CI provider confirmation (GitHub Actions / EAS Workflows / Bitrise)
7. E2E framework approval (recommendation: Maestro for managed Expo)
8. Authorisation to add dev dependencies: `@testing-library/react-native`, `@testing-library/jest-native`, `msw`, `@types/jest`
9. Confirmation on the `vizzeon` vs `kreatifa` domain split (Bug #10) — which is current?
10. Confirmation whether school registration is intentionally missing Step 2 (Bug #60) or it's broken

QA commitments

- QA will **not touch source code**. All fixes are dev team execution.
- QA provides: per-bug writeups (this document), the test plan (`TEST_PLAN.md`), and will deliver a regression test suite once the dev deps are approved and Sprint 1 fixes deployed to staging.
- QA will verify each fix against the "Fix recommendation" stated under each bug.

This document was assembled from five independent code-analysis rounds on commit `622729f` branch `main`. Strategy and test infrastructure details are in `TEST_PLAN.md` at the project root. Both documents are gitignored as QA artifacts — delivered manually, not via PR.